

UNINFORMED SEARCH

BFS • UCS • DFS • DLS • IDS

Russell & Norvig | Bengaluru → Hyderabad classroom example

How an AI agent chooses the next state to explore

Learning Objectives

- Understand uninformed (blind) search.
- Explain BFS, UCS, DFS, DLS and IDS.
- Trace the algorithms using Bengaluru → Hyderabad.
- Compare completeness, optimality, time and space.
- Connect each algorithm with its supporting data structure.

What Is Uninformed Search?

Search without a heuristic estimate.

INPUT

Initial state • actions • transition model • goal test • path cost

NO HEURISTIC

The algorithm does not know which state is closer to the goal.

CENTRAL QUESTION

Among the frontier nodes, which one should I expand next?

The five algorithms give different answers to this question.

Search Vocabulary

State

A situation/location.

Node

A search-tree record.

Frontier

Discovered nodes waiting for expansion.

Depth

Number of actions from start.

b

Branching factor.

d

Depth of shallowest goal.

m

Maximum depth.

$g(n)$

Cost from start to n .

Common Problem: Bengaluru → Hyderabad

Same problem; different search strategy.

START

Bengaluru

INTERMEDIATE

Anantapur / Chitradurga

GOAL

Hyderabad

Example route costs

Bengaluru → Anantapur → Hyderabad = $3 + 7 = 10$

Bengaluru → Chitradurga → Kadapa → Hyderabad = $4 + 5 + 4 = 13$

Goal: find a route from the initial state to the goal state.

BFS — Breadth-First Search

Expand the shallowest unexpanded node first.

RULE

Search level-by-level: depth $0 \rightarrow 1 \rightarrow 2 \rightarrow \dots$

DATA STRUCTURE

FIFO Queue — First In, First Out.

- Explores all nodes at one depth before going deeper.
- With identical step costs, BFS finds a shallowest and optimal solution.
- Time: $O(b^d)$ Space: $O(b^d)$

UCS — Uniform-Cost Search

Expand the node with the lowest cumulative path cost.

RULE

Choose the smallest $g(n)$, where $g(n)$ is total cost from the start.

DATA STRUCTURE

Priority Queue — lowest-cost node first.

BFS vs UCS

BFS asks: “Which is shallowest?” UCS asks: “Which is cheapest so far?”

A deeper route can be preferred if its total cost is lower.

UCS Example — Cost Matters

ROUTE A

Bengaluru → Anantapur → Hyderabad

Cost = $3 + 7 = 10$

ROUTE B

Bengaluru → Chitradurga → Kadapa → Hyderabad

Cost = $4 + 5 + 4 = 13$

UCS prefers Route A because $10 < 13$.

Number of steps is not the deciding factor; cumulative path cost is.

DFS — Depth-First Search

Expand the deepest unexpanded node first.

RULE

Go deep along one branch before backtracking.

DATA STRUCTURE

LIFO Stack — Last In, First Out.

- Can use very little memory compared with BFS.
- Not complete in general; can get trapped down a deep branch.
- Not optimal.
- Time: $O(b^m)$ Space: $O(bm)$

DLS — Depth-Limited Search

DFS with a maximum depth limit l .

RULE

Search like DFS, but never go beyond depth l .

PURPOSE

Prevents DFS from going indefinitely deep.

CUTOFF

The limit was reached. A solution may exist deeper.

FAILURE

The allowed search space is exhausted with no solution.

IDS — Iterative Deepening Search

Repeatedly run DLS with increasing limits.

LIMIT 0

DLS
CUTOFF

LIMIT 1

DLS
CUTOFF

LIMIT 2

DLS
CUTOFF

LIMIT 3

DLS
GOAL FOUND

0 → 1 → 2 → 3 → ...

Combines BFS-like shallow-solution behavior with DFS-like low memory.

Properties at a Glance

ALGORITHM	COMPLETE?	OPTIMAL?	TIME	SPACE
BFS	Yes*	Yes*	$O(b^d)$	$O(b^d)$
UCS	Yes†	Yes†	Cost-dependent	Cost-dependent
DFS	No‡	No	$O(b^m)$	$O(bm)$
DLS	No‡	No	$O(b^l)$	$O(bl)$
IDS	Yes*	Yes*	$O(b^d)$	$O(bd)$

* finite b ; identical step costs for optimality. † appropriate positive step-cost conditions. ‡ not complete in general.

The Most Important Difference

BFS

SELECTS:
Shallowest node

QUEUE

DFS

SELECTS:
Deepest node

STACK

UCS

SELECTS:
Lowest $g(n)$

PRIORITY QUEUE

Depth $\rightarrow$ BFS / DFS | Cost $\rightarrow$ UCS

DLS = DFS + limit | IDS = repeated DLS

Easy Memory Trick

BFS

SHALLOW

Queue

UCS

CHEAP

Priority Queue

DFS

DEEP

Stack

DLS

DEEP + LIMIT

Stack + limit

IDS

0 → 1 → 2 → 3

Repeated DLS

BFS = breadth • UCS = cost • DFS = depth • DLS = depth cap • IDS = repeated depth cap

FINAL TAKEAWAY

Same problem. Different node-selection rule.

BFS → SHALLOWEST

UCS → CHEAPEST

DFS → DEEPEST

DLS → LIMIT

IDS → REPEAT LIMITS